DBMS ACADEMIC MINI PROJECT • PRESENTATION SCRIPT & VIVA GUIDE

Campus Fix 🛠️

Campus Maintenance & Complaint Management System — 3NF/BCNF Schema & ACID Transactions

ACADEMIC COURSE
Database Management Systems (DBMS)

PROJECT GUIDE
Prof. P. R. Nerkar

DATABASE ENGINE
MySQL 8.0+ / PostgreSQL / IndexedDB

👥 Team Roster & Individual Contributions

SPEAKER & ROLL NO.	ASSIGNED ROLE	SPECIFIC DBMS CONTRIBUTION & DEFENSE DOMAIN	DURATION
Sarth Santosh Pande Roll No. 28	Team Lead & Architect	Project Motivation, Problem Statement, System Architecture & Role-Based Access Control (RBAC)	0:00 - 2:00
Takshak Rahul Sirsat Roll No. 30	Database Modeler	Relational Schema, E-R Modeling, 8 Entities, 1:M & 1:1 Cardinalities, PK/FK & Cascades	2:00 - 4:00
Manthan Vinod Dhamal Roll No. 35	Normalization Lead	Relational Theory (1NF → 2NF → 3NF → BCNF) & Anomaly Elimination Proofs	4:00 - 6:00
Ali Asgar Yusuf Ali Roll No. 40	Query Optimization Lead	Advanced SQL: Multi-Table JOINs, GROUP BY Aggregations, SLA Metrics & Live SQL Studio	6:00 - 8:00
Rohit Sudhir Karpe Roll No. 53	Transaction Engineer	Automated Database Triggers, ACID Properties, Master Activity Audit Trail & Conclusion	8:00 - 10:00
Optional 6th Member / Shared Live Demo Lead	UI & System Integration	Live End-to-End Workflow: Ticket Creation → Admin Dispatch → Real-Time SQL Stream → Feedback	10:00 - 11:30

⏱️ Presentation Flow Timeline (10 – 12 Minutes Total)

0:00 – 2:00	2:00 – 4:00	4:00 – 6:00	6:00 – 8:00	8:00 – 10:00	10:00 – 12:00
Sarth (28) Intro & RBAC	**Takshak (30)** ER & Schema	**Manthan (35)** 3NF & BCNF	**Ali Asgar (40)** SQL Studio	**Rohit (53)** Triggers & ACID	**All Team** Viva Defense

🎙️ Speaker Cue Cards (Spoken Scripts & Live Actions)

👨‍🎓 Speaker 1: Sarth Santosh Pande
Roll No. 28 • Team Lead & System Architect

⏱️ 0:00 – 2:00 (2 Mins)

🗣️ **WHAT TO SAY (SPOKEN DIALOGUE):**

"Respected guide Prof. P. R. Nerkar Sir, and dear friends. A very good morning to all of you.

*Today, our team is proud to present our DBMS Mini Project titled: **'Campus Fix — Campus Maintenance and Complaint Management System'**.*

In an educational campus, hundreds of infrastructure issues occur daily—such as power failures, laboratory equipment faults, and network downtime. Traditional maintenance relies on paper registers or phone calls, leading to ticket loss, zero staff accountability, and no audit trails.

*To solve this, we designed **Campus Fix**: a fully relational, normalized database system. We implemented **Role-Based Access Control (RBAC)** across three primary tiers: **Students** (who lodge geotagged tickets), **Staff Technicians** (assigned by department), and **Administrators** (who dispatch work orders and monitor SLAs).*

*Our backend database dump is fully production-ready in MySQL and PostgreSQL. I now hand over to **Takshak** to explain our Relational Database Design and Entity-Relationship model."*

🖥️ **WHAT TO SHOW ON SCREEN:**

Open `index.html` in browser. Point to the top brand banner *"Campus Fix • Guided by Prof. P. R. Nerkar"*. Briefly highlight the dashboard cards and role switcher dropdown.

🔄 **HANDOVER LINE:**

"Now, Takshak will explain the E-R Diagram, our 8 relational entities, and integrity constraints."

Speaker 2: Takshak Rahul Sirsat

Roll No. 30 • Database Modeler

⏱ 2:00 – 4:00 (2 Mins)

🗣️ **WHAT TO SAY (SPOKEN DIALOGUE):**

*"Thank you, Sarth. Respected Sir, I will walk you through the architectural foundation of our database: the **Entity-Relationship Model and Relational Schema**.*

*Our database consists of **8 core relational tables**:*

1. `departments` *: Divisions like Electrical, IT, Civil, Sanitation, and Labs.*
2. `users` *: Role-based user records linked with departmental foreign keys.*
3. `categories` *: Maps issue types with target SLA resolution hours.*
4. `complaints` *: The central operational entity storing ticket numbers, reporter IDs, locations, priorities, and statuses.*
5. `assignments` *: Administrator staff allocation orders with instructions.*
6. `complaint_updates` *: Chronological technician work notes.*
7. `feedback` *: 1-to-5 star student satisfaction reviews.*
8. `activity_logs` *: Immutable transaction logs.*

Cardinalities & Integrity:

- *Department to Categories is 1:M; Category to Complaints is 1:M.*
- *Student to Complaints is 1:M, but Complaint to Feedback is a **strict 1:1 relationship** enforced via a* `UNIQUE` *constraint on* `complaint_id`*.*
- *We enforce* `ON DELETE CASCADE` *on dependent child records,* `ON DELETE SET NULL` *on* `assigned_to`*, and a* `CHECK (rating ≥ 1 AND rating ≤ 5)` *constraint on student ratings."*

🖥️ **WHAT TO SHOW ON SCREEN:**

Click the 🗄️ **DBMS Studio** button in the navigation bar. Stay on Tab 1 (**E-R Diagram**). Point out the primary keys (PK), foreign keys (FK), and the 1:1 relation connecting `complaints` and `feedback`.

🔁 **HANDOVER LINE:**

"I now invite Manthan to present our Normalization proofs from 1NF to BCNF."

Speaker 3: Manthan Vinod Dhamal
Roll No. 35 • Relational Theory & Normalization Lead

⏱️ 4:00 – 6:00 (2 Mins)

🗣️ **WHAT TO SAY (SPOKEN DIALOGUE):**

*"Thank you, Takshak. Respected Sir, in database engineering, poor design leads to Insertion, Update, and Deletion anomalies. My focus was mathematically proving that our schema satisfies **Third Normal Form (3NF)** and **Boyce-Codd Normal Form (BCNF)**.*

*• **1NF (Atomic Attributes):** We eliminated all repeating groups and comma-separated tags. Every column holds indivisible values, and every table has a defined primary key.*
*• **2NF (No Partial Dependencies):** A table in 1NF is in 2NF if all non-prime attributes are fully functionally dependent on the entire primary key. In assignments, technician details are not stored directly; they reference `user_id`, eliminating partial key dependencies.*
*• **3NF (No Transitive Dependencies):** 3NF mandates that no non-prime attribute depends transitively on a primary key ($X \to Y \to Z$). Initially, `complaint_id` → `category_id` → `dept_name`. If department names changed, update anomalies would occur across complaints. We decomposed this into independent tables: `departments`, `categories`, and `complaints`.*
*• **BCNF (Strict Superkeys):** In the `feedback` table, both `feedback_id` and `complaint_id` act as candidate keys, satisfying BCNF with zero redundancy."*

🖥️ **WHAT TO SHOW ON SCREEN:**

Inside the DBMS Studio modal, switch to Tab 3 (**Normalization Proofs**). Show the interactive cards comparing the *"Before (Non-Normalized)"* anomaly vs. *"After (3NF Decomposed)"* structure.

🔁 **HANDOVER LINE:**

"Now, Ali Asgar will demonstrate our SQL queries and live SQL sandbox execution."

Speaker 4: Ali Asgar Yusuf Ali
Roll No. 40 • Query Optimization & SQL Studio Lead

⏱ 6:00 – 8:00 (2 Mins)

🗣️ **WHAT TO SAY (SPOKEN DIALOGUE):**

*"Thank you, Manthan. Respected Sir, my contribution focused on crafting optimized SQL queries for reporting and building our **Live SQL Query Sandbox**.*

Let me run our core queries live:
*1. **Multi-Table Relational JOIN:** We join 4 relational tables (`complaints` , `users` , `categories` , and `departments`) to retrieve student names and departments without storing redundant text.*
*2. **Department Performance & SLA Aggregation:***
`SELECT d.dept_name, COUNT(c.complaint_id) AS total, SUM(CASE WHEN c.status IN ('Resolved','Closed') THEN 1 ELSE 0 END) AS resolved FROM departments d LEFT JOIN categories cat ON d.department_id = cat.department_id LEFT JOIN complaints c ON cat.category_id = c.category_id GROUP BY d.dept_name;`
Here we use a `LEFT OUTER JOIN` with `GROUP BY` and conditional `SUM(CASE ...)` so departments with zero complaints are not lost.

Our built-in SQL Sandbox also supports live DDL queries like `SHOW TABLES` and `DESCRIBE complaints` with real-time execution duration in milliseconds."

🖥️ **WHAT TO SHOW ON SCREEN:**
Switch to Tab 2 (**Live SQL Sandbox**). Click Preset Query 1 → Click ▶ **Run SQL [Ctrl+Enter]**. Show the resulting data table. Then click Preset Query 2 and run it live to show department counts.

🔁 **HANDOVER LINE:**
"I now hand over to Rohit to present our Database Triggers, ACID transactions, and audit logs."

Speaker 5: Rohit Sudhir Karpe

Roll No. 53 • Transaction & Automation Engineer

⏱ 8:00 – 10:00 (2 Mins)

🗣️ **WHAT TO SAY (SPOKEN DIALOGUE):**

*"Thank you, Ali. Respected Sir, to guarantee that business logic is enforced at the database layer, we engineered **Automated Database Triggers** and managed **ACID Transactions**.*

*We implemented **3 essential triggers** in our MySQL DDL:*
1. `trg_after_complaint_assigned`: An `AFTER INSERT` trigger on `assignments`. As soon as an admin dispatches a technician, the trigger automatically flips complaint status from 'Pending' to 'Assigned' and links `assigned_to`.
2. `trg_auto_close_on_feedback`: An `AFTER INSERT` trigger on `feedback` that auto-transitions the ticket to 'Closed' and stamps `closed_at = NOW()`.
3. `trg_audit_status_update`: An `AFTER UPDATE` trigger on `complaints` logging all status transitions into `activity_logs`.

***ACID Guarantees:** Operations execute inside atomic transactions (`BEGIN TRANSACTION ... COMMIT`). If any table insertion fails, changes rollback immediately, ensuring Consistency and Isolation.*

To conclude: Campus Fix successfully demonstrates relational modeling, normal forms, triggers, and transactions in a real-world institutional platform. We are now ready for viva questions."

🖥️ **WHAT TO SHOW ON SCREEN:**

Switch to Tab 4 (**DDL Scripts & Triggers**). Scroll down to the trigger definitions. Then click the top ⚡ **Live SQL** drawer button to show live SQL logs being captured in real-time.

🔁 **HANDOVER LINE:**

"Sir, our entire team is now open for your viva voce questions. Thank you!"

Speaker 6 (or Shared): Dedicated Live Workflow Demo

Frontend Integration & Live System Demo Lead

⏱ 10:00 – 11:30 (1.5 Mins)

🗣️ **WHAT TO SAY (LIVE DEMO WALKTHROUGH):**

"Sir, let me demonstrate our complete relational lifecycle in action:
*1. **Lodge Complaint:** As student Sarth Pande, I report a broken projector in Seminar Hall 1. The RDBMS engine executes an atomic `INSERT INTO complaints`.*
*2. **Live SQL Drawer:** Notice the Live SQL Drawer on the right immediately logs the SQL statement.*
*3. **Admin Dispatch:** We switch to Admin Prof. P. R. Nerkar and assign technician Amit Verma. Trigger 1 executes instantly, transitioning the ticket to 'Assigned'.*
*4. **Resolution & Feedback:** The technician submits work notes, marks it 'Resolved', and the student rates it 5 stars. Trigger 2 closes the ticket automatically."*

🖥️ **LIVE SCREEN STEPS:**

Click "+ Report Issue" → Submit ticket → Switch profile to Admin → Assign technician → Show status badge update in real-time.

🎯 Academic Viva Voce Defense Sheet (Questions & Model Answers)

Q1: "Why is the complaints table in 3NF and not just 2NF?" — Answered by: Manthan (Roll 35)

Model Answer: "Sir, in 2NF, transitive dependencies can still exist. In our domain, `department_name` depends on `category_id`, which depends on `complaint_id`. If we kept `dept_name` inside `complaints`, any department update would require updating hundreds of complaint records. Decomposing it into `departments` and `categories` removes transitive dependencies and satisfies 3NF."

Q2: "What happens if an admin tries to delete a department? Will tickets be lost?" — Answered by: Takshak (Roll 30)

Model Answer: "Sir, we have set `ON DELETE RESTRICT` on the `categories` → `departments` foreign key. The database rejects the deletion of a department if active categories or complaints reference it. On `users.department_id`, we set `ON DELETE SET NULL` so staff accounts are preserved."

Q3: "Why did you use database triggers instead of handling status updates in frontend JavaScript?" — Answered by: Rohit (Roll 53)

Model Answer: "Sir, frontend validation can be bypassed or disabled. Database triggers execute directly in the RDBMS engine. Whether a record is inserted through our web UI, a mobile client, or direct SQL script in MySQL Workbench, the trigger guarantees that status transitions and audit logs are recorded 100% consistently."

Q4: "Can you explain the difference between WHERE and HAVING in your SQL queries?" — Answered by: Ali Asgar (Roll 40)

Model Answer: "Sir, the `WHERE` clause filters individual records before aggregation. `HAVING` filters group-level results after `GROUP BY`. For example, to filter complaints lodged in 2026, we use `WHERE YEAR(created_at) = 2026`. To find departments with more than 5 complaints, we use `GROUP BY dept_name HAVING COUNT(*) > 5`."

Q5: "How is the 1:1 relationship between complaints and feedback enforced?" — Answered by: Takshak / Sarth

Model Answer: "Sir, each complaint should have at most one feedback review. We enforce this at the schema level by placing a `UNIQUE` constraint on `feedback.complaint_id` in addition to its Foreign Key. Any attempt to insert a second review for the same ticket triggers a duplicate key error."